Cheat Sheet: Getting Started with MCP

Estimated time needed: 10 minutes

This module explored the basics of creating and configuring MCP servers. Use this cheat sheet to quickly review and refresh the concepts you learned.

MCP basics

Topic	Description
What is MCP?	Model Context Protocol (MCP) is an open-source standard for enabling AI applications with the capabilities of external systems. Using MCP, large language models (LLMs) like ChatGPT can execute tasks (e.g., sending an email), retrieve data (e.g., call information from the local file system), and perform specialized workflows (e.g., iterative code review).
Analogy	A simple analogy for MCP is to think of it like a USB port for AI applications. MCP provides a standardized interface that lets AI applications connect with external systems, just like how USB offers a universal way for devices to connect and communicate.
Why MCP?	Standardized integration: Set a standard/protocol for connecting AI to external systems, consistent and simple development patterns improve understandability and scalability.Interoperability: Wide support for different clients and AI models, and it is modular.Security: Uses OAuth 2.0 and token-based authentication.Minimizes hallucinations: Connects to external systems that are kept up-to-date, ensuring clean, accurate data, which reduces incorrect responses.Agentic workflow support: Enables AI agents for multi-step, dynamic tasks.

MCP architecture

Topic	Description
Three-layer architecture	MCP uses bidirectional communication using three message types in **JSON-RPC 2.0** (Communication format). These messages are sent over STDIO or HTTP, transports you will learn in the next module.
Initialization	1. InitializationClient sends an `initialize` request with protocol version and capabilities.Server responds with its capabilities.Client sends `initialized` notification to complete the handshake.2. OperationBidirectional communication begins.Client discovers and invokes tools, resources, and prompts provided by the server.The server can send notifications, logging messages, and other requests to the client.3. ShutdownClient sends a `shutdown` request, server responds and closes the connection.
Multi-server client	The host process manages multiple MCP client instances.Each client handles its own sessions with a server (lifecycles and capabilities).The host process aggregates capabilities and routes tool results to the correct client.

Interaction with an MCP server

Use the `FastMCP` library to create a simple `Client` that can call tools from an MCP server. First, import the client and the transport classes.

```
from fastmcp import Client
from fastmcp.client.transports import StreamableHttpTransport, StdioTransport
```

Then, create the transports. You can use either **STDIO** or **HTTP**:

```
stdio_transport = StdioTransport(
    command = "npx",
    args=["-y", "@upstash/context7-mcp"]
)
http_transport = StreamableHttpTransport(
    url="https://mcp.context7.com/mcp"
)
```

Here, you'll connect each transport to the same Context7 MCP Server. The transports are easily configurable to connect to any MCP server that supports that type of transport.

Then create a client for each transport:

```
stdio_client = Client(stdio_transport)
http_client = Client(http_transport)
```

- Finally, you can open an **asynchronous context manager** for the client and call the tools using `client.list_tools()`
    - This lists all tools with their name, description, and metadata (includes input schema).
- You can also use the `client.call_tool("tool_name", {"key": "value"})` method to call a tool with input parameters.

```
async with stdio_client as client:
    # List of tools the server provides
    tools = await client.list_tools()
    # Call the tool with the given parameter(s)
    response = await client.call_tool("resolve-library-id", {
        "libraryName": "fastmcp"
    })
```

You can also use the HTTP client:

```
async with http_client as client:
    # List of tools the server provides
    tools = await client.list_tools()
    # Call the tool with the given parameter(s)
    response = await client.call_tool("resolve-library-id", {
        "libraryName": "fastmcp"
    })
```

Author(s)

[Joshua Zhou | Data Scientist Intern @ IBM]

[Joseph Santarcangelo | Data Scientist @ IBM]

